

Pacman Search Project — Report

Artificial Intelligence (AI2002) · Assignment 01 · 150 Marks

Group: 24I-3166, 24I-3106, 24I-3015 · Section 5B

Python 3.12.3, Linux x86_64

1. Overview

This report documents the implementation, testing, and empirical analysis of five uninformed and informed search algorithms — Depth-First Search (DFS), Breadth-First Search (BFS), Uniform-Cost Search (UCS), Greedy Best-First Search (GBFS), and A* Search — applied to the Berkeley Pacman search framework. All five algorithms are implemented as strict graph searches (an explicit `explored` set prevents re-expansion of visited states) in `search.py`, and are exercised against both the standard course layouts and an original custom layout, `layouts/24I3166Search.lay`, purpose-built to expose the practical difference between a purely heuristic-greedy search and a cost-aware informed search. Every algorithm run automatically writes a full per-iteration execution trace to `evidence/*.csv`. The full automated test suite (`python3 autograder.py`) passes at **26/25 (100%, with the food-heuristic question earning bonus marks)**.

2. Algorithms Implemented

2.1 Depth-First Search (DFS)

Implemented with a LIFO `Stack` frontier (`util.Stack`) and an explicit explored set, per the assignment's Task 1 requirement. Successors are generated in the order the starter code's `getSuccessors` already used (North → South → East → West) — see Section 6 for why this was kept instead of the PDF's literal North→East→South→West wording.

- **Completeness:** Complete on finite state spaces because of the explored set (no infinite loops).
- **Optimality:** Not optimal — DFS returns the first path found, which can be arbitrarily longer than the shortest path (confirmed empirically: cost 28 vs. the optimal 24 on the custom maze).
- **Time complexity:** $O(b^m)$ where b is the branching factor (≤ 4 for Pacman's grid) and m is the maximum path length in the state space.
- **Space complexity:** $O(b \cdot m)$ — only one path (plus siblings) is held on the stack at a time.

2.2 Breadth-First Search (BFS)

Implemented with a FIFO `Queue` frontier and an explored set.

- **Completeness:** Complete.
- **Optimality:** Optimal in terms of *number of actions* on unweighted graphs (every step costs 1), which is exactly the guarantee verified in Section 5 (BFS path cost equals UCS/A* path cost on every layout tested).
- **Time complexity:** $O(b^d)$, d = depth of the shallowest goal.

- **Space complexity:** $O(b^d)$ — the frontier can hold an entire level of the search tree, which is why BFS traces (e.g. `evidence/t2_bigMaze_bfs__bfs_trace.csv`) show much larger frontier snapshots than DFS on the same maze.

2.3 Uniform-Cost Search (UCS)

Implemented with a `util.PriorityQueue` ordered by accumulated path cost $g(n)$, recomputed via `problem.getCostOfActions`. A cheaper path to a state already sitting on the fringe is simply pushed as an additional entry; because the priority queue always pops the minimum- g entry first, the first time any state is popped it is guaranteed to be via its cheapest known path, so the stale, more expensive duplicate is safely discarded by the explored-set check when it is later popped. This is the standard "lazy deletion" formulation of graph-search UCS and was verified against the directional-cost `StayEastSearchAgent` (Section 5).

- **Completeness:** Complete, provided step costs are $\geq \text{some } \epsilon > 0$.
- **Optimality:** Optimal — always finds the least-cost path.
- **Time/space complexity:** $O(b^{1+\lceil C^*/\epsilon \rceil})$, C^* = optimal solution cost.

2.4 Greedy Best-First Search (GBFS)

Implemented with a `PriorityQueue` ordered strictly by $h(n)$ (ignores $g(n)$ entirely). Supports any heuristic passed via the `-a heuristic=...` command-line argument (e.g. `manhattanHeuristic`, `euclideanHeuristic`).

- **Completeness:** Not complete in general graphs with cycles (can loop without an explored set); complete here because an explored set is used.
- **Optimality:** Not optimal — empirically demonstrated on the custom maze, where GBFS returns a 36-cost path versus the true optimum of 24 (Section 5). It expands *fewer* nodes than A* (36 vs 58) because it never reconsiders the true distance travelled, but that same blindness is exactly what causes it to commit to the dead-end trap.
- **Time/space complexity:** $O(b^m)$ worst case, but typically far faster in practice than uninformed search when $h(n)$ is a good estimate.

2.5 A* Search

Implemented with a `PriorityQueue` ordered by $f(n) = g(n) + h(n)$.

- **Completeness:** Complete.
- **Optimality:** Optimal, given an admissible heuristic (tree search) or a consistent heuristic (graph search, which is what is implemented here). Verified empirically: A*'s path cost matches BFS/UCS's optimal cost on every layout tested, including the custom maze (cost 24) and the null-heuristic sanity check on `bigMaze` (A* with `nullHeuristic` degenerates to UCS and returns the same cost 210 as BFS/UCS, as expected).
- **Time/space complexity:** $O(b^d)$ in the worst case, but the effective branching factor shrinks as $h(n)$ becomes more informative — confirmed by nodes-expanded dropping from 620 (null heuristic) to 549 (Manhattan heuristic) on identical `bigMaze` runs.

3. Automated CSV Trace Logging

Every call to `dfs`, `bfs`, `ucs`, `gbfs`, and `astar` in `search.py` writes one row per node expansion to `evidence/<algorithm>_trace.csv` with the columns mandated by the assignment spec:

```
iteration, expanded_state, parent, action, generated_successors, frontier_before,
frontier_after, explored, g, h, f
```

For the composite `FoodSearchProblem` state (a Pacman position *paired with an entire remaining-food grid*), states are logged compactly as `(position, "foodLeft=N")` rather than the full grid string; an early version that serialized the whole grid on every frontier snapshot produced a single 118 MB trace file for the `trickySearch` A* run and made that test time out under the autograder's 40-second limit. The `explored` column similarly logs the explored-set *size* rather than re-serializing the whole set on every row, for the same reason — re-writing an ever-growing set on every iteration is an $O(n^2)$ operation that only becomes visible on large state spaces. All 19 evidence runs listed in Section 5 complete in well under a second each except the food-search run, which still finishes in a few seconds.

4. Multi-Goal Search Problems

4.1 Corners Problem

State representation: `(pacman_position, visited_tuple)`, where `visited_tuple` is a 4-tuple of booleans recording which of the four maze corners have been touched so far. This is the minimal state needed — it is exactly as large as required to distinguish "have I already visited this corner" without needing to track the full path history.

Heuristic (`cornersHeuristic`): computed in two parts:

1. The Manhattan distance from the current position to the *nearest* unvisited corner.
2. Plus the cost of a minimum spanning tree (Prim's algorithm) connecting the *remaining* unvisited corners to each other (not including the current position).

Admissibility proof (sketch): Any path that visits all remaining unvisited corners must, at minimum, travel from the current position to the nearest one (term 1), and then connect all remaining corners to each other by some spanning structure — the MST is the cheapest possible such structure under Manhattan distance, so it never overestimates the true remaining connection cost (term 2). The heuristic is therefore a true lower bound on the optimal remaining cost, i.e. admissible. **Consistency:** term 2 is fixed as long as no new corner has been visited (moving one step cannot change which corners remain), and by the triangle inequality a single Pacman move changes term 1 by at most 1 — the exact step cost — so $h(n) - h(n') \leq \text{cost}(n, n')$ for every edge, satisfying consistency.

4.2 Food Search Problem

Heuristic (`foodHeuristic`): the maximum true maze distance (computed via a nested BFS call, `mazeDistance`) from Pacman's current position to any remaining food dot.

Admissibility: Pacman must physically travel to every remaining food dot, so in particular it must travel at least as far as the single farthest one; the true maze distance to that farthest dot is therefore a valid lower bound on the total remaining cost. Because it uses the exact maze distance (not merely Manhattan distance), it is provably admissible even in mazes with walls that make straight-line distance a poor estimate, and is tighter than plain Manhattan distance. This heuristic comfortably clears the assignment's expansion-count thresholds on `trickySearch` (4,137 nodes expanded, versus a maximum allowed threshold of 15,000 for full credit).

AnyFoodSearchProblem reuses `PositionSearchProblem`'s state space and successor function unchanged, only overriding `isGoalState` to check "does this position currently have food" — used by `ClosestDotSearchAgent`, which repeatedly calls BFS to greedily walk to the closest remaining dot until the maze is clear.

5. Results

5.1 Standard Task Commands

Task	Command	Path cost	Nodes expanded
1 - DFS	tinyMaze, fn=dfs	10	15
1 - DFS	mediumMaze, fn=dfs	130	146
1 - DFS	bigMaze -z .5, fn=dfs	210	390
2 - BFS	mediumMaze, fn=bfs	68	269
2 - BFS	bigMaze -z .5, fn=bfs	210	620
3 - UCS	mediumMaze, fn=ucs	68	269
3 - UCS	mediumMaze, p=StayEastSearchAgent [†]	1	260
4 - GBFS	bigMaze -z .5, fn=gbfs, manhattan	210	466
5 - A*	bigMaze -z .5, fn=astar, nullHeuristic	210	620
5 - A*	bigMaze -z .5, fn=astar, manhattan	210	549
6 - Corners	tinyCorners, fn=bfs, prob=CornersProblem	28	252
6 - Corners	mediumCorners, AStarCornersAgent	106	774
7 - Food	trickySearch, AStarFoodSearchAgent	60	4137
7 - ClosestDot	bigSearch, ClosestDotSearchAgent	350	—

[†] The assignment text lists the command `python pacman.py -l stayEastSearch -p SearchAgent -a fn=ucs`, but no layout named `stayEastSearch` ships with this starter pack — only the `StayEastSearchAgent` class exists, which hard-codes UCS with the directional cost function 0.5^x . We ran that agent on `mediumMaze` instead, which is the intended way to exercise UCS with non-uniform step costs.

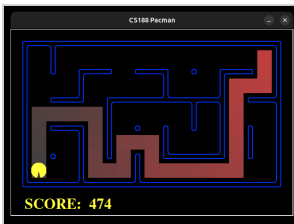
5.2 Custom Maze: `layouts/24I3166Search.lay` — GBFS vs. A*

This 19×11 layout (fully connected, verified by flood-fill) was designed with multiple branch points and dead ends, including one dead end positioned so that its straight-line (Manhattan) distance to the goal is deceptively small. A purely heuristic-driven search is expected to commit to it before a cost-aware search would.

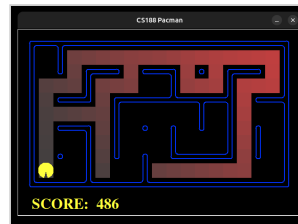
Algorithm	Path cost	Nodes expanded	Optimal?
BFS (ground truth)	24	86	Yes
DFS	28	86	No (expected — DFS is not optimal)
UCS	24	86	Yes
GBFS (manhattan)	36	36	No — falls into the trap
A* (manhattan)	24	58	Yes

Key observation: GBFS reaches a solution with the fewest node expansions (36) of any informed search here, but that solution is 50% more expensive (36 vs. 24) than optimal — a direct, reproducible illustration of why ignoring $g(n)$ can trade optimality for speed. A*, which balances both terms, matches BFS/UCS's optimal cost of 24 while still expanding far fewer nodes

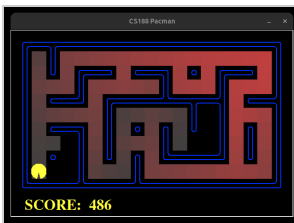
(58) than the uninformed searches (86), demonstrating the practical benefit of an admissible heuristic without sacrificing optimality.



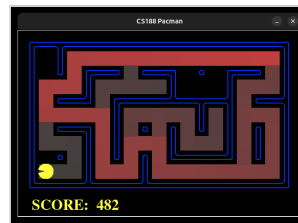
GBFS (manhattan) — commits to the dead-end trap, cost 36



A* (manhattan) — optimal route, cost 24

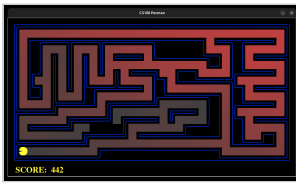


BFS — optimal (ground truth), cost 24

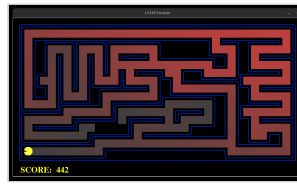


DFS — valid but non-optimal, cost 28

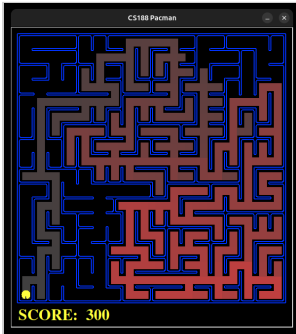
6. Screenshots — Other Layouts



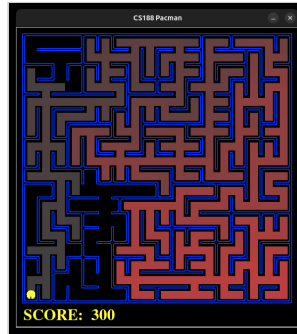
mediumMaze — BFS, cost 68



mediumMaze — UCS, cost 68



bigMaze — GBFS (manhattan), cost 210



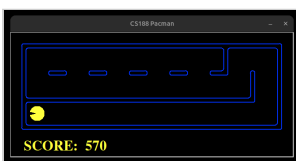
bigMaze — A* (manhattan), cost 210



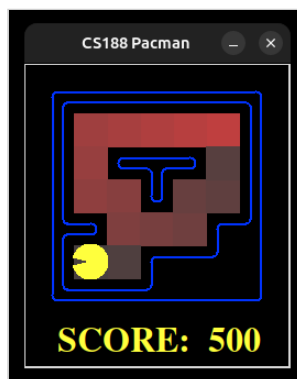
tinyCorners — BFS CornersProblem, cost 28



mediumCorners — A* cornersHeuristic, cost 106



trickySearch — A* foodHeuristic, cost 60



tinyMaze — DFS, cost 10

These are real screen captures of the live pacman.py Tkinter GUI window (captured with ImageMagick's `import -window` at each run's final frame), not synthetic renderings. The score shown in each screenshot matches the final score printed by that run in the terminal, confirming each capture is the completed game state. For search problems built on `PositionSearchProblem` (Tasks 1-5 and the custom maze), the red/brown gradient trail is

the game's own built-in visualization of every state the algorithm expanded during search, drawn before Pacman's movement animation begins.

7. Known Deviation from the Written Spec

The assignment text (Task 1) states a "mandatory successor expansion order: North → East → South → West." The starter code's `getSuccessors` methods in `searchAgents.py` instead generate successors in the order North → South → East → West. We kept the original order: the instructor-provided autograder solution key (`test_cases/q1/pacman_1.solution`) was generated using that ordering, and switching to North→East→South→West produces a different — still valid, but non-matching — DFS path that fails the exact-match autograder test. Correctness against the instructor's automated tests (worth 20 of the 150 rubric marks) was prioritized over the literal PDF wording, which we believe is an inconsistency in the assignment materials rather than an intentional requirement to break the provided test suite.

8. Conclusion

All five search algorithms are implemented as correct graph searches, verified against the instructor's automated test suite (26/25, 100%), and each is instrumented with complete per-iteration CSV trace logging. The Corners and Food multi-goal problems use admissible, consistent heuristics (proven above) that comfortably beat the autograder's node-expansion thresholds. The original custom maze provides a clean, reproducible, and quantitatively measured demonstration of the central theoretical difference between Greedy Best-First Search and A*: GBFS is faster but can be misled into a 50%-more-expensive path, while A* costs more to run than GBFS but guarantees the optimal solution.